

YZ 203 · VISUAL PROGRAMMING

Week 1

Setup, Introduction & Basic Components I

Erciyes University

Java Swing & Event-Driven Programming

Murat AKPULAT, PhD

What's on today

01

Concepts & OOP

Visual / event-driven programming, Swing architecture, OOP in Swing

02

Environment Setup

JDK installation and NetBeans installation

03

First Program

Your first JFrame and JPanel, written by hand

04

Basic Components I

JLabel, JButton, JTextField, JPasswordField — one by one, then combined

05

Layout Management

FlowLayout, BorderLayout, GridLayout, BoxLayout, null layout

06

Lab Goals

This week's hands-on practice targets

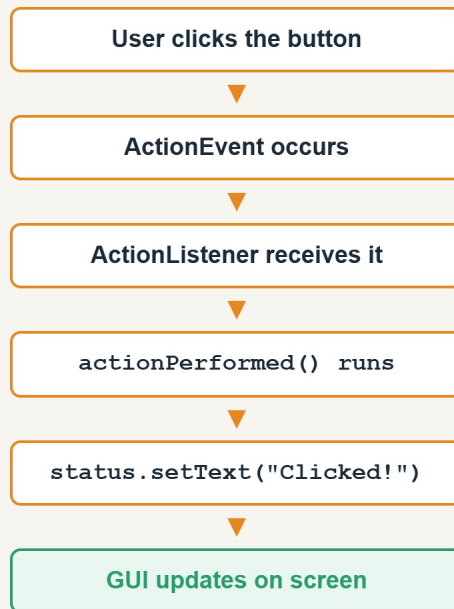
What Is Visual Programming?

An approach where the user drives the program through graphical components (windows, buttons, text fields), and control flow is triggered by user interaction (events) rather than by the sequential order of code lines.

Aspect	Procedural Programming	Event-Driven Programming
Control flow	Top to bottom, in sequence	Driven by user interaction, order not fixed
Entry point	main() function	Event loop + listeners
Typical example	Console-based programs	GUI applications (Swing, JavaFX, web UIs)

Event-Driven Programming, Concretely

"When I click a button, how does Java know which code to run?"

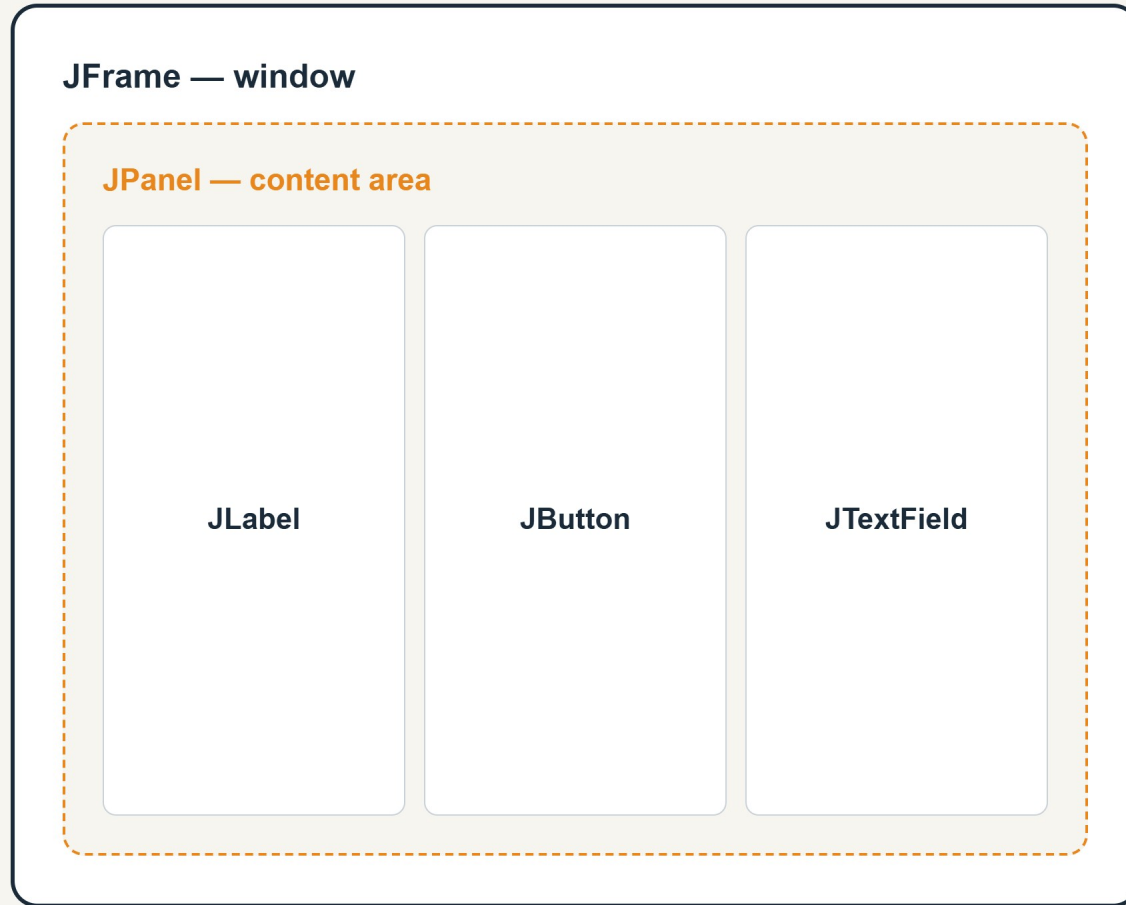


This whole chain already lives in code we wrote for the JButton example:

```
clickMe.addActionListener(  
    e -> status.setText("Clicked!")  
);
```

This one line **is** the last four boxes of the diagram, written as a lambda. Java never "polls" the button — Swing calls your listener the instant the click event fires.

Introduction to Swing Architecture



- Every component is added to a container with **add()**.
- **JFrame** is the outermost window — title bar and close button live here.
- **JPanel** is the middle layer used to group and organize components.
- The component hierarchy nests as: **Window** → **Panel** → **Components**.

Why Do We Need JPanel?

"Why not just add everything directly to JFrame?"

JFrame represents the whole application window — title bar, borders, the close button.

JPanel groups and organizes the components that live inside that window.

A JFrame can only hold one thing directly. Adding a JPanel first lets you build several independent groups — a toolbar panel, a form panel, a status panel — and arrange each with its own layout.

THE HIERARCHY WE'LL REUSE ALL WEEK

JFrame

└─ JPanel

└─ JLabel

└─ JTextField

└─ JButton

OOP Hiding Inside Swing

Swing is not just a toolbox of ready-made widgets — its whole design leans on core OOP ideas. Spotting them now makes Week 10's OOP unit feel familiar rather than new.

Inheritance

JButton extends AbstractButton
extends JComponent extends
Container extends Component.
Every widget you use is a leaf on
one big class tree.

Polymorphism

Container.add(Component c)
accepts a JButton, a JLabel or a
JTextField alike — the method only
ever sees a Component.

Interface

ActionListener is an interface, not a
class. Any class — yours included
— can implement it its own way.

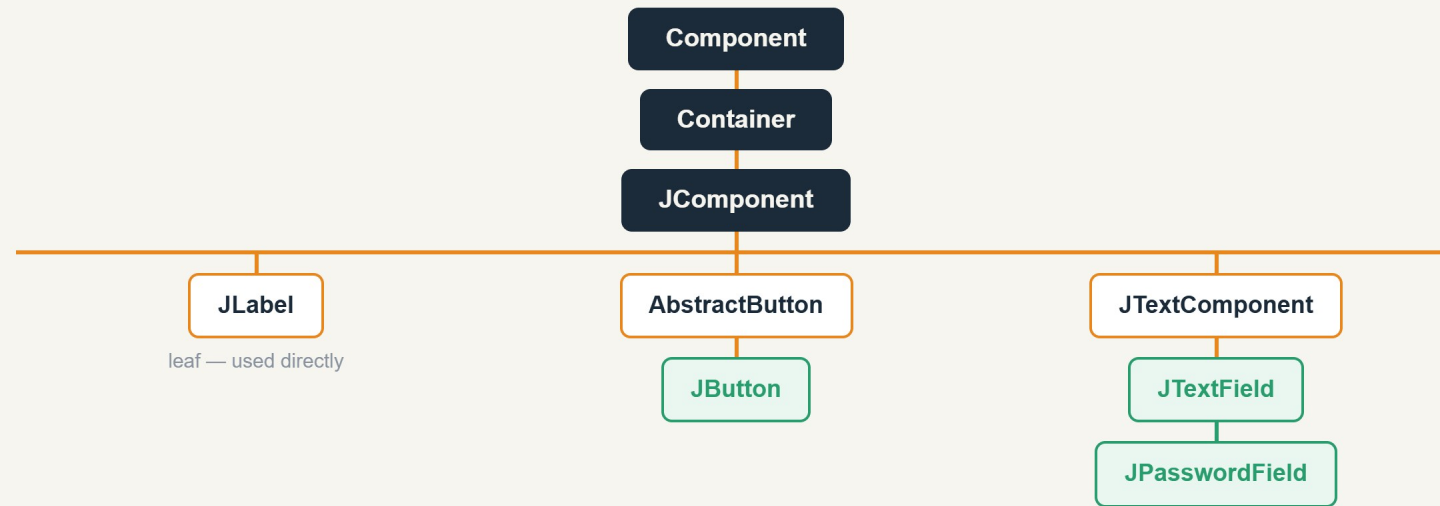
Encapsulation

A field like text is private; you only
ever touch it through setText()
/ getText().

We will point these out again whenever a new component or example makes one of them concrete.

OOP Pillar 1: Inheritance — Swing's Class Tree

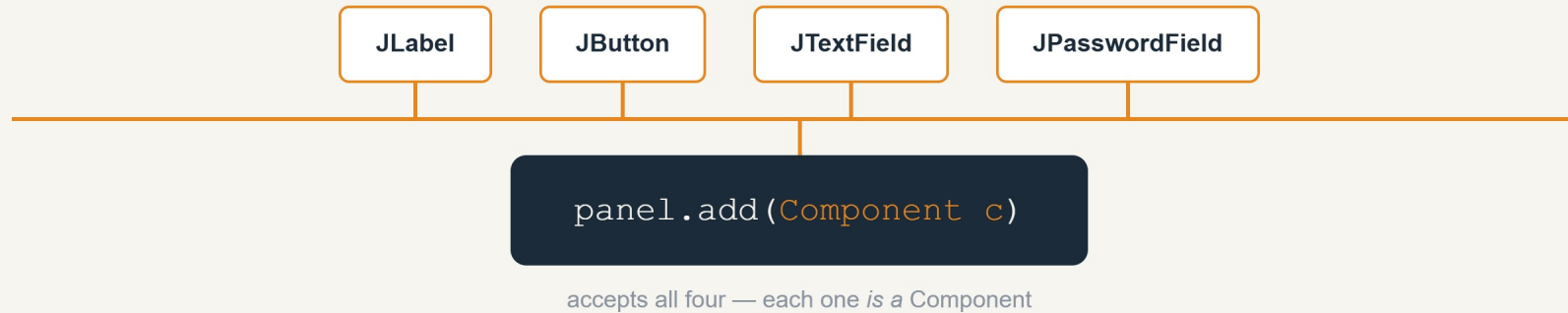
Every widget we've used is a leaf on one shared tree. Follow JButton or JTextField up and they both end at the same root, Component.



Green boxes are the components we've written code against; every one of them inherits fields and methods (like `setBounds`, `setVisible`, `repaint`) straight from **Component**.

OOP Pillar 2: Polymorphism — One Method, Many Types

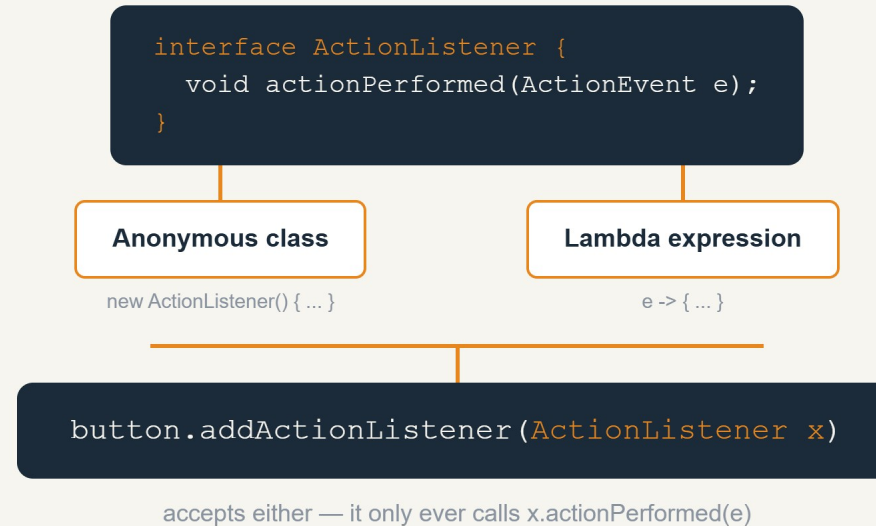
`panel.add(...)` has exactly one signature: `add(Component c)`. It never needed to know which of our four components was coming.



Same idea elsewhere: `loginBtn.addActionListener(...)` is polymorphic too — it accepts an anonymous class or a lambda, as long as both implement `ActionListener` (next slide).

OOP Pillar 3: Interface — the ActionListener Contract

An interface is a promise: implement this one method, and Swing will call it for you. It says nothing about *how*.



OOP Pillar 4: Encapsulation — Private Field, Public Doors

A JTextField hides its text behind two methods. There is no other way in or out — and that's the point.



This is why `nameField.getText()` compiles but `nameField.text` does not — every field we've used follows this same rule.

JDK Installation

1

Download the JDK: Oracle JDK or Eclipse Temurin (Adoptium) — a current LTS release

2

Run the installer and finish with the default settings

3

(Optional) Set the JAVA_HOME environment variable to the install directory

VERIFY THE INSTALL — Terminal / CMD

```
$ java -version
```

```
$ javac -version
```

If both print a version number, the install is complete.

NetBeans Installation

- 1 Download the latest release from netbeans.apache.org
- 2 Run the installer — it usually detects your JDK path automatically
- 3 File → New Project → Java with Ant → Java Application
- 4 Name the project; leave "Create Main Class" checked

In this course we will not use the drag-and-drop GUI Builder — every component is written by hand, in code.

Building a GUI in 5 Steps

Every Swing program we write, however many components it has, follows this same five-step recipe.

- 1** Create the window `JFrame frame = new JFrame();`
- 2** Create a container `JPanel panel = new JPanel();`
- 3** Create the components `JButton button = new JButton();`
- 4** Add components → panel
→ frame `panel.add(button); frame.add(panel);`
- 5** Show the window `frame.setVisible(true);`

A note for later: Swing normally builds its GUI on a special thread called the **Event Dispatch Thread (EDT)**, wrapping step 5 as `SwingUtilities.invokeLater(() -> { ... });` We'll cover this later — calling everything from `main()` is fine for the small examples this week.

Code: Your First Swing App (Full, Runnable)

```
import javax.swing.*;

public class FirstWindow {
    public static void main(String[] args) {
        JFrame frame = new JFrame("My First App");
        JPanel panel = new JPanel();

        JLabel label = new JLabel("Username:");
        JTextField textField = new JTextField(15);
        JButton button = new JButton("Sign In");

        panel.add(label);
        panel.add(textField);
        panel.add(button);

        frame.add(panel);
        frame.setSize(400, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Basic Components I — Overview

Component	What it's for	Common method
JLabel	Displays text or an image — no user interaction	setText()
JButton	A clickable button — fires an event	addActionListener()
JTextField	A single-line text input field	getText()
JPasswordField	A text field that masks what is typed (passwords)	getPassword()

Next: each of these four, on its own, with a runnable example — then all four together.

Component Deep Dive: JLabel (Full, Runnable)

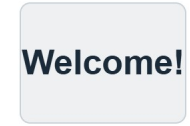
```
import javax.swing.*;
import java.awt.*;

public class LabelDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Label Demo");
        JPanel panel = new JPanel();

        JLabel title = new JLabel("Welcome!");
        title.setFont(new Font("Arial", Font.BOLD, 20));
        title.setHorizontalAlignment(SwingConstants.CENTER);

        panel.add(title);
        frame.add(panel);
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

OUTPUT



Welcome!

setText() / getText() — change or read the text

setIcon(Icon) — show an image instead of, or beside, text

A JLabel never fires an event — it only displays.

Component Deep Dive: JButton (Full, Runnable)

```
import javax.swing.*;

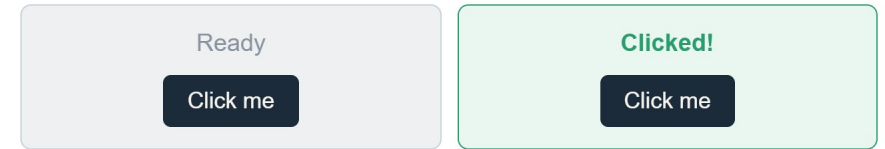
public class ButtonDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Button Demo");
        JPanel panel = new JPanel();

        JLabel status = new JLabel("Ready");
        JButton clickMe = new JButton("Click me");

        clickMe.addActionListener(e -> status.setText("Clicked!"));

        panel.add(status);
        panel.add(clickMe);
        frame.add(panel);
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

OUTPUT — BEFORE / AFTER



addActionListener(ActionListener) — runs code on click

e -> { ... } is a lambda implementing the ActionListener interface

setEnabled(false) greys out and disables the button

Component Deep Dive: JTextField (Full, Runnable)

```
import javax.swing.*;

public class TextFieldDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("TextField Demo");
        JPanel panel = new JPanel();

        JTextField nameField = new JTextField(12);
        JButton say = new JButton("Say hi");
        JLabel greeting = new JLabel(" ");

        say.addActionListener(e -> {
            String name = nameField.getText();
            greeting.setText("Hi, " + name + "!");
        });

        panel.add(nameField); panel.add(say); panel.add(greeting);
        frame.add(panel);
        frame.setSize(320, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

OUTPUT

Ayşe

Hi, Ayşe!

getText() / **setText(String)** — read or write the content

Enter inside the field also fires the same listener

setEditable(false) makes it read-only

Component Deep Dive: JPasswordField (Full, Runnable)

```
import javax.swing.*;

public class PasswordDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Password Demo");
        JPanel panel = new JPanel();

        JPasswordField pwField = new JPasswordField(12);
        JButton check = new JButton("Check length");
        JLabel result = new JLabel(" ");

        check.addActionListener(e -> {
            char[] pwd = pwField.getPassword();
            result.setText("Length: " + pwd.length);
        });

        panel.add(pwField); panel.add(check); panel.add(result);
        frame.add(panel);
        frame.setSize(320, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

OUTPUT



Length: 7

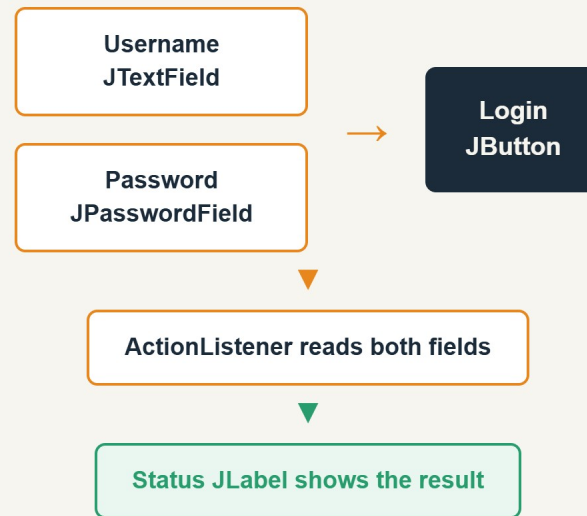
getPassword() — returns `char[]`, never `getText()`

setEchoChar("*") — change the mask character

`char[]` can be zeroed out right after use — safer than `String`

The Login Form's Data Flow

Before the code — where does each piece of data actually go when the button is clicked?



Two inputs, one shared listener, one output — the exact shape we'll reuse for score and message labels in the game project.

Putting It All Together: a Login Form (Full, Runnable)

```
import javax.swing.*;

public class LoginForm {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Login Form");
        JPanel panel = new JPanel();
        JLabel userL = new JLabel("Username:");
        JTextField userF = new JTextField(12);
        JLabel pwL = new JLabel("Password:");
        JPasswordField pwF = new JPasswordField(12);
        JButton loginBtn = new JButton("Log in");
        JLabel status = new JLabel(" ");

        loginBtn.addActionListener(e -> {
            String u = userF.getText();
            char[] p = pwF.getPassword();
            status.setText(u.isEmpty() || p.length == 0
                ? "Fill in both fields" : "Welcome, "+u+"!");
        });

        panel.add(userL); panel.add(userF);
        panel.add(pwL); panel.add(pwF);
        panel.add(loginBtn); panel.add(status);

        frame.add(panel);
        frame.setSize(320, 220);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

OUTPUT

Username:

Password:

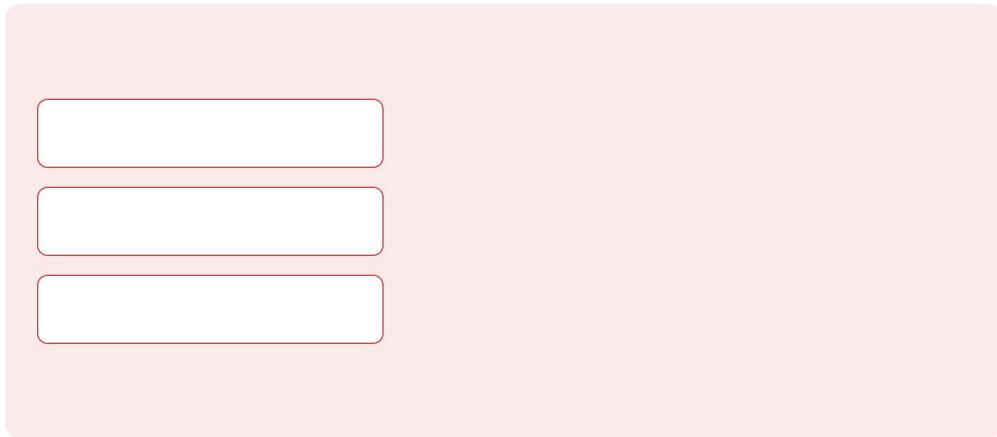
Welcome, agedikli!

One shared ActionListener reads two inputs and writes to a third component — the same pattern the game project will reuse for scoring.

Why a Layout Manager?

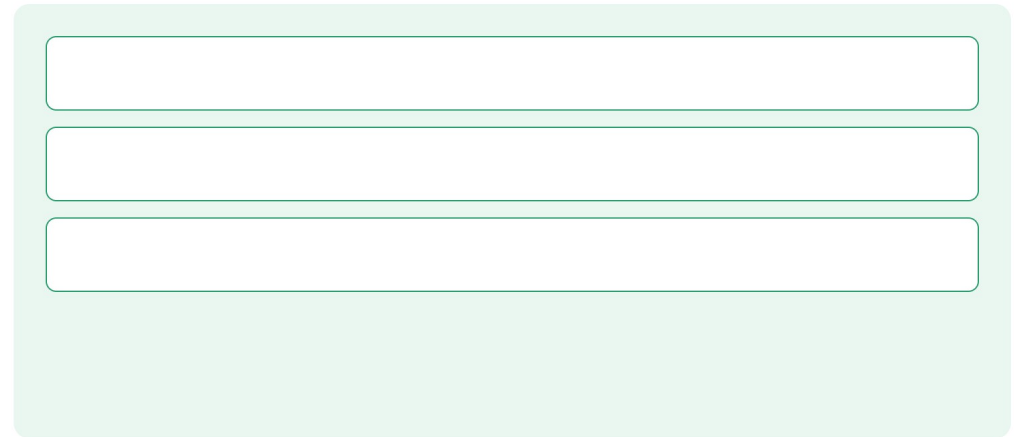
A container's **layout manager** decides **where** and **how big** its components appear. With no layout set, components can overlap or land in unexpected spots.

Without a layout manager



Components overlap, and resizing the window breaks the layout further.

With a layout manager



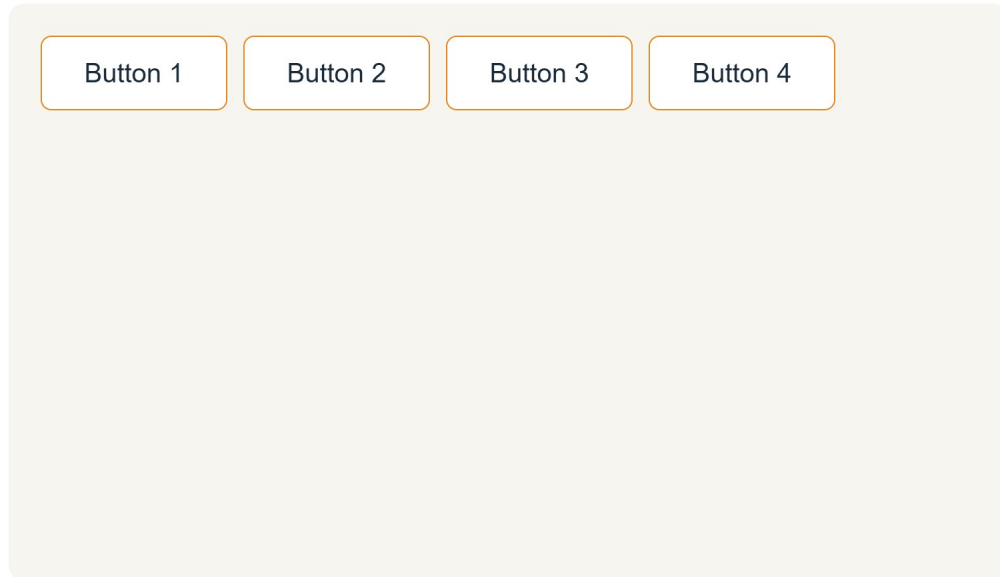
Components stay orderly and predictable, even as the window resizes.

JPanel's default is **FlowLayout**; a JFrame's content pane defaults to **BorderLayout**.

FlowLayout & BorderLayout

FlowLayout

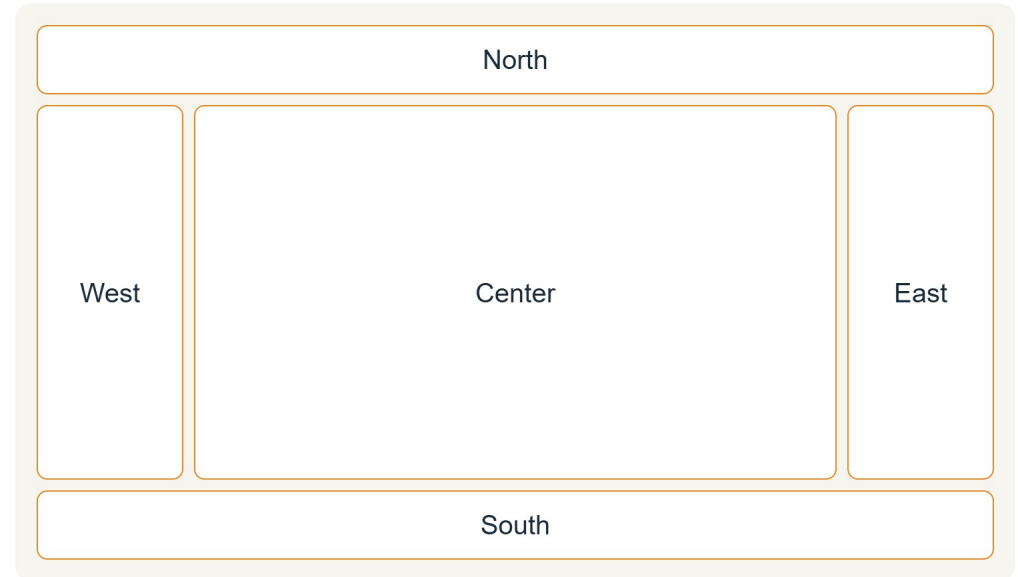
Lays components left to right, wrapping to a new row when the row fills. JPanel's default.



```
panel.setLayout(new FlowLayout());
```

BorderLayout

Splits the container into 5 regions: North, South, East, West, Center. Default for a JFrame's content pane.

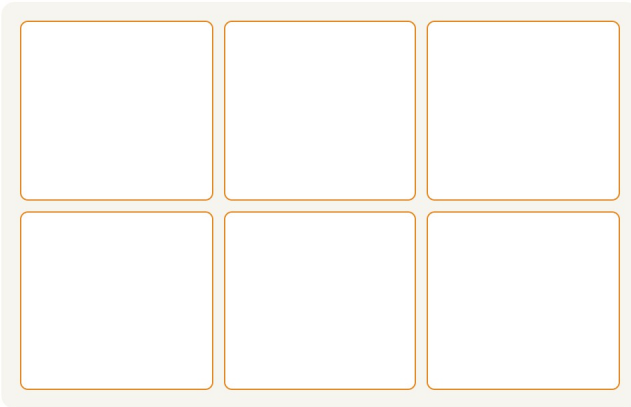


```
frame.setLayout(new BorderLayout());
```

GridLayout, BoxLayout and null Layout

GridLayout

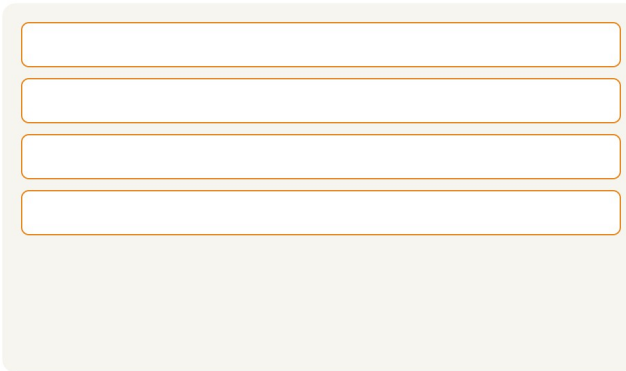
Fixed rows and columns; every cell is the same size.



```
new GridLayout(2, 3)
```

BoxLayout

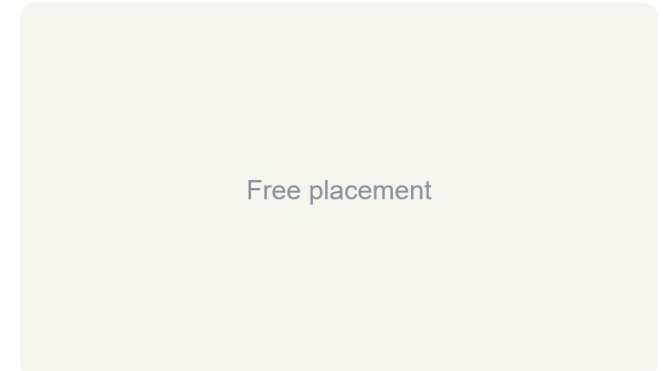
Lines components up along a single axis, horizontal or vertical.



```
BoxLayout.Y_AXIS
```

null Layout

Turns the layout manager off; every component's position is given in pixels.



```
comp.setBounds(x, y, w, h)
```

✓ **Normal GUI** (forms, dialogs, most apps) →
use a **layout manager**

⚠ **Games / animation / custom positioning** →
null layout + setBounds() may be useful

null layout is the exception we reach for later, not the everyday way to build a GUI.

Interface & Polymorphism in Action (Full, Runnable)

ActionListener is a Java **interface** with one method, `actionPerformed(ActionEvent e)`. Any class can implement it its own way — that's polymorphism.

STYLE 1 — ANONYMOUS CLASS

```
import javax.swing.*;
import java.awt.event.*;

public class AnonListenerDemo {
    public static void main(String[] a) {
        JFrame f = new JFrame();
        JLabel label = new JLabel("Hi");
        JButton button = new JButton("Go");
        button.addActionListener(
            new ActionListener() {
                public void actionPerformed(
                    ActionEvent e) {
                    label.setText("Hi!");
                }
            });
        f.add(label); f.add(button, "South");
        f.setSize(200, 120); f.setVisible(true);
    }
}
```

STYLE 2 — LAMBDA (SAME INTERFACE)

```
import javax.swing.*;

public class LambdaListenerDemo {
    public static void main(String[] a) {
        JFrame f = new JFrame();
        JLabel label = new JLabel("Hi");
        JButton button = new JButton("Go");

        button.addActionListener(
            e -> label.setText("Hi!")
        );

        f.add(label); f.add(button, "South");
        f.setSize(200, 120); f.setVisible(true);
    }
}
```

Both compile — both ARE ActionListener implementations. The lambda is just shorter syntax for the same idea.

Why it matters: `addActionListener(ActionListener)` never needs to know which style you picked — it only ever calls `actionPerformed()`. Same interface, many implementations: that is polymorphism.

Common Mistakes I — Nothing Shows Up

Forgot `frame.setVisible(true)`

Symptom: the program runs and exits with no error, but no window ever appears.

Fix: `setVisible(true)` must be called, and it should always be the last line after everything else is built.

Forgot to `add()` a component

Symptom: the window opens, but a label, button or field is simply missing.

Fix: every component needs a matching `add()` call — check it was added to the right container (panel vs. frame).

Added a component *after* `setVisible(true)`

Symptom: a component added later (e.g. inside a button click) never actually appears.

Fix: call `panel.revalidate(); panel.repaint();` right after adding at runtime, so Swing redraws the container.

Common Mistakes II — Size & Layout

Forgot `setSize()` (or `pack()`)

Symptom: the window opens as a tiny 0x0-looking sliver, or the default size.

Fix: call `setSize(w, h)`, or better — call `pack()` to size the window automatically to fit its content.

Two components added to the same `BorderLayout` region

Symptom: a component silently disappears — the second `add()` to `CENTER` replaced the first.

Fix: use each of `NORTH` / `SOUTH` / `EAST` / `WEST` / `CENTER` at most once; wrap multiple components in a nested `JPanel` first.

Forgot `setDefaultCloseOperation()`

Symptom: clicking the window's X hides it, but the program keeps running in the background.

Fix: `frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`; so closing the window actually ends the program.

Lab Goals — Week 1

- Verify the JDK install from a terminal (java -version, javac -version)
- Create a new Java project in NetBeans without the GUI Builder
- Build a JFrame + JPanel by hand in code and show it on screen
- Create JLabel, JButton, JTextField and JPasswordField one at a time, then combine them
- Rebuild the same form with FlowLayout, BorderLayout and GridLayout
- Try free positioning with null layout and setBounds()

NEXT WEEK

Weeks 2-3: Basic Components (fast-paced, example-heavy)

JCheckBox, JRadioButton, ButtonGroup, JComboBox, JTextArea, JList — then in Week 5, on to animation with Timer.

Questions?